Chapter 11: Functions

Suppose we need the average of three numbers, more than once, for different numbers each time:

```
total = 2 + 4 + 9;
average1 = total / 3;
print(average1);
total = 10 + 20 + 30;
average2 = total / 3;
print(average2);
```

This prints `5.0`, then `20.0`. The same three statements, typed twice, differing only in which numbers go in. A **function** lets us name that computation once and supply different numbers, called **arguments**, each time we use it:

```
function average(a, b, c) {
    total = a + b + c;
    return total / 3;
};
print(average(2, 4, 9));
print(average(10, 20, 30));
```

Both print the same two results as before. `average` is defined once; calling it with different arguments runs the same three statements against whatever values were supplied that time.

Writing a Function

A function value is written with the `function` keyword, a parenthesized, comma-separated list of parameter names, and a brace-delimited body:

```
double = function(x) { return x + x };
print(double(21));
```

This prints `42`. `function(x) { return x + x }` is an expression, a value, the same as a list literal or an object literal is a value. It can be assigned to a name, as above, which is how `double` becomes callable.

Giving a function a name this often is common enough to earn its own shorthand:

```
function triple(x) { return x + x + x };
print(triple(7));
```

This prints `21`. `function triple(x) { return x + x + x }` is **syntactic sugar**: it means exactly `triple = function(x) { return x + x + x };`, and produces the identical abstract syntax tree (AST). Nothing past parsing ever sees the named form; by the time the evaluator runs, every function in this chapter arrived the same way, as a value assigned to a name. The named form exists because `function name(...) { ... }` reads better than `name = function(...) { ... };` does, not because it is a different kind of thing.

A function can take any number of parameters, including none:

```
function announce() { print("starting") };
function add3(a, b, c) { return a + b + c };
```

Calling a function with the wrong number of arguments is a runtime error. `add3(1, 2)` and `add3(1, 2, 3, 4)` both fail; only `add3(1, 2, 3)` matches the three parameters `a`, `b`, and `c` were declared with.

A function value is represented the same way a Vertex object is: a dict. The evaluator already requires every object key to be a string, so a function value marks itself with one key no Vertex program could ever write – not a string at all, just the Python value `None`, paired with a number that counts up by one every time a function literal is evaluated. No object literal can ever produce that key, so nothing a Vertex program builds can ever be mistaken for a function, and no two function values, however identical their parameters and their body, ever carry the same one.

This is worth pausing on, because it is new. Every earlier chapter asked Python directly what type a value was and got Vertex's own answer for free: a Vertex number is a Python `int` or `float`, a string a `str`, and so on, one Vertex concept per Python type, with nothing more to check. That works only as long as Vertex's value space and Python's stay in lockstep. This chapter is the first time two different Vertex concepts, objects and functions, wanted to share one Python representation, a dict, and Python's type system alone had nothing left to say about which was which. The actual guarantee above comes from a narrower, deliberately chosen invariant – object keys must be strings – not from Python's types as such.

A stronger alternative exists, not built here: make every Vertex value an explicit pair, `(type, data)` – `("integer", 5)`, `("object", {...})`, `("function", {...})` – so Vertex's own type names do the tagging, instead of being inferred after the fact from Python's. That would make this chapter's trick unnecessary rather than merely simpler, and it pays off further: porting to a host language without Python's reflection (a direct struct in a language like Go), and toward a self-hosted interpreter, where only Vertex's own data structures are available to represent anything at all. The cost is real – every operation would need to tag and untag values explicitly, instead of asking a question Python already answers – which is why this chapter does not take it on. It is a likely early enhancement in a future phase of this course, not a retroactive rewrite of what already works.

return, and the Value a Call Produces

`return` stops a function's body immediately and hands a value back to whoever called it:

```
function classify(n) {
    if (n < 0) { return "negative" };
    if (n == 0) { return "zero" };
    return "positive";
};
print(classify(-5));
print(classify(0));
print(classify(5));
```

This prints `negative`, `zero`, then `positive`. Once `return` runs, nothing else in the function's body runs, however deeply nested inside `if` statements or blocks the `return` itself was.

A function does not have to return anything:

```
function announce() { print("starting") };
print(announce());
```

This prints `starting`, then `null`. **`null`** is the value a function call has when its body finishes without an explicit `return`, or when `return` runs with no expression after it. It is a value like any other – it can be stored, compared, and printed – used specifically to mean "there is no value here."

```
print(null);
print(null == null);
print(type(null));
```

This prints `null`, then `true`, then `null`. Arithmetic, ordering comparisons, and indexing all reject `null`, the same way they already reject a boolean or a list where a number was expected.

A function call used only for its side effect does not need its result stored anywhere:

```
function announce() { print("starting") };
announce();
```

A bare call like this is a complete statement on its own. In fact, any expression may stand alone as a statement; its value is simply computed and discarded. A call is the common case of that, since it can do something – print, mutate, recurse – while its result goes unused, but it is not the only one: an assignment used as a statement is just as effectful, changing a binding whether or not anything reads the value the assignment itself produced, and `input()` consumes a line of input the same way regardless of what becomes of its result.

Local Variables and Call Frames

A name assigned inside a function body belongs to that call alone:

```
function f(x) {
    y = x * 2;
    return y + 1;
};
print(f(5));
```

This prints `11`. `y` exists only while `f(5)` is running; it is not visible to whatever called `f`, and a second call does not see the first call's `y`, even calling the same function twice in a row:

```
function f(x) {
    doubled = x * 2;
    return doubled;
};
a = f(3);
b = f(100);
print(a);
print(b);
```

This prints `6`, then `200`. Each call gets a fresh, private place to put its own local variables – a **call frame** – that disappears once the call returns. The next section says precisely what a call frame is.

Dynamic Scoping

A function body can read a name it never declared itself:

```
function showGreeting() { print(greeting) };
function englishCaller() { greeting = "Hello"; showGreeting() };
function frenchCaller() { greeting = "Bonjour"; showGreeting() };
englishCaller();
frenchCaller();
```

This prints `Hello`, then `Bonjour`. `showGreeting` never assigns `greeting` itself. Each caller does, right before calling `showGreeting`, and `showGreeting` reads whichever one happened to be active. This is **dynamic scoping**: a name a function does not bind itself resolves by looking at whoever called it, not by looking at wherever the function happened to be written down. (1)

The mechanism is the same `$PARENT` chain every environment has used since Chapter 6. Calling a function builds a new environment whose `$PARENT` is the environment active right at the call site:

```
englishCaller();
```

When this runs, `englishCaller`'s own call frame binds `greeting`, then calls `showGreeting`. `showGreeting`'s new call frame has `$PARENT` set to `englishCaller`'s frame – the environment active at the moment of that specific call – not the top-level environment where `showGreeting` itself was defined. Looking up `greeting` inside `showGreeting` walks that `$PARENT` link, finds it in `englishCaller`'s frame, and stops.

This has a real limitation. A function's own value does not retain any record of where it was defined:

```
function makeAdder(a) {
    return function(b) { return a + b };
};
y = makeAdder(1)(2);
```

This raises an error: `Unknown identifier: a`. `makeAdder(1)` builds a call frame with `a` bound to `1`, and returns a brand-new function value from inside it – but that returned function retains only its own parameters and body, nothing about the frame it was built in. By the time it is called, with `(2)`, a new call frame is built whose `$PARENT` is wherever *that* call happened, which is nowhere near `a`. The next chapter has a function value retain the environment active where it was defined, and changes a call's `$PARENT` to that retained environment instead of the one active at the call site – to fix it.

Recursion

A function can call itself with no special support:

```
function factorial(n) {
    if (n == 0) { return 1 };
    return n * factorial(n - 1);
};
print(factorial(6));
```

This prints `720`. Dynamic scoping already explains why this works. `factorial`'s own name is bound once, at the top level, where the function was defined. Every call – however deep the recursion – builds a new frame whose `$PARENT` chains back through the call that made it, and that chain always eventually reaches back to the top level, where `factorial` itself still lives. Looking up `factorial` inside its own body finds it the same way `showGreeting` found `greeting`: by walking `$PARENT` links outward until the name turns up.

Calls as Expressions

A call is a factor, the same as a number, a list literal, or an object literal, and it can appear anywhere an expression can:

```
function square(x) { return x * x };
print(1 + square(3) + 1);
```

This prints `11`. A function that returns a boolean can be used directly as a condition:

```
function isPositive(x) { return x > 0 };
if (isPositive(5)) {
    print("5 is positive");
} else {
    print("5 is not positive");
};
```

A call's result can be indexed, read by field, or called again immediately, chained the same way `[]` and `.` already chained with each other:

```
function pair() { return [10, 20] };
print(pair()[1]);
function record() { return {"value": 99} };
print(record().value);
function makeAdder() { return function(a, b) { return a + b } };
print(makeAdder()(3, 4));
```

This prints `20`, then `99`, then `7`. `makeAdder()(3, 4)` is two calls: `makeAdder()` with no arguments, producing a function value, immediately called again with `(3, 4)`. Nothing about this example runs into the limitation from the previous section, because the inner function here reads only its own parameters, `a` and `b`, never a name from an enclosing call.

Types and Supported Operations

Operation	Function behavior
`type(function(x) { return x })`	Returns `"function"`
`string(f)`, `print(f)`	Displays as `<function>`
`f == g`	`true` only when `f` and `g` are the same function value
Arithmetic, ordering comparisons	Type error
`length`, `head`, `tail`, `slice`, indexing	Type error

Operation	`null` behavior
`type(null)`	Returns `"null"`
`string(null)`, `print(null)`	Displays as `null`
`null == null`	`true`
Arithmetic, ordering comparisons	Type error

A function compares equal only to itself. An object or a list compares structurally, field by field or element by element, but a function's counting marker is a number no two function values ever share – so the same structural comparison that would otherwise apply to any dict never gets past that one field for two different functions. Nothing function-specific had to be added to equality for this to hold; the generic rule already in place for objects reports them as different on its own. Vertex has no operation for inspecting a function's parameters or body from inside the language.

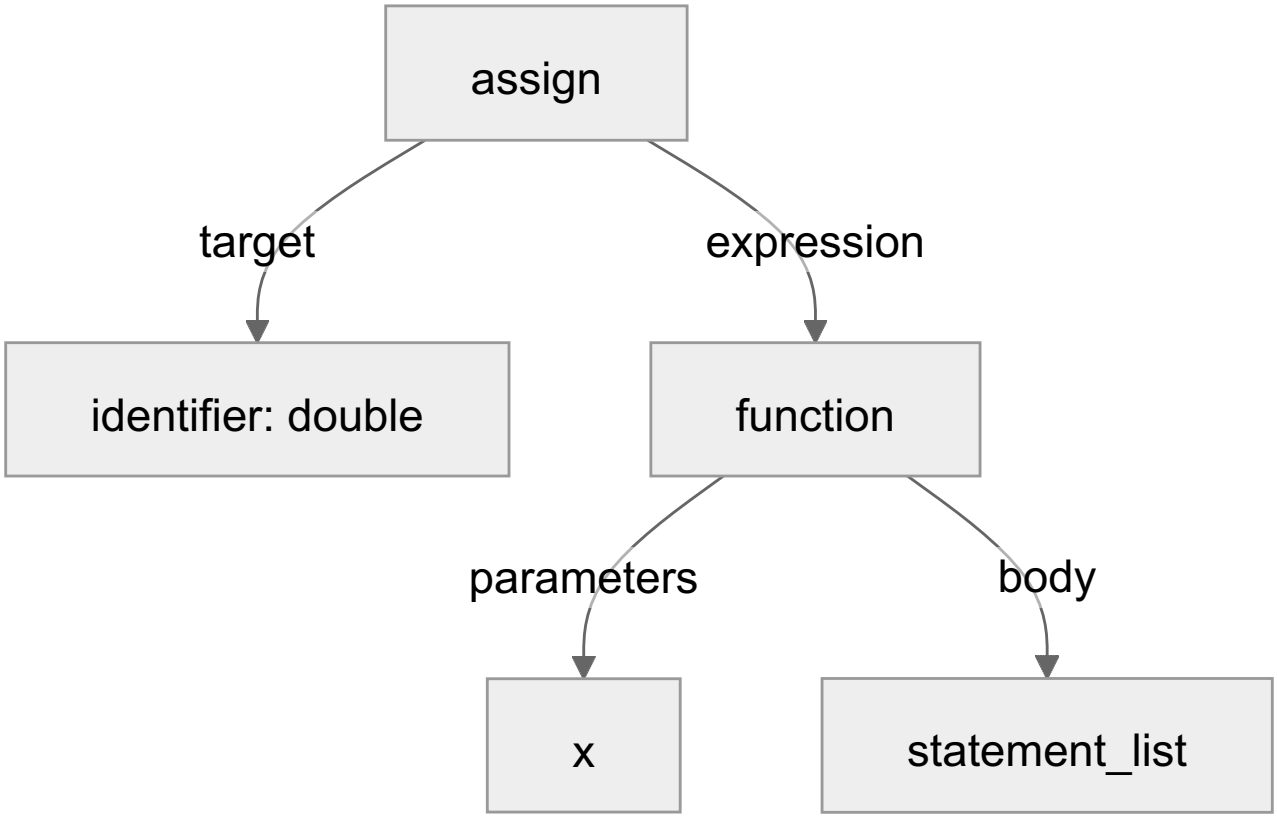
Parsing the New Syntax

Three keywords arrive this chapter: `function`, `return`, and `null`. The grammar for a function value and a call is:

```
function_expression ::= "function" "(" [ identifier { "," identifier } ] ")" block
function_definition ::= "function" identifier
                        "(" [ identifier { "," identifier } ] ")" block
return_statement    ::= "return" [ expression ]
call_suffix         ::= "(" [ expression { "," expression } ] ")"
```

`function_expression` is parsed as a factor, alongside list and object literals. `function_definition`, the named form, resolves at parse time into an `assign` node wrapping that same function-expression AST – the evaluator never sees a `function_definition` node at all.

`call_suffix` extends the same suffix chain `[` and `.` already use. Since Chapter 9, that chain has produced `complex` nodes for indexing; a call produces a `call` node instead, since it carries an argument list rather than a single index, but it shares the same chain, which is what lets suffixes mix freely in any order and combination: `funcs[0](x)`, `makeAdder(1)(2)`, and `record().value` are each one pass through this one loop.



Assignment moves into the expression grammar this chapter, rather than staying its own kind of statement: `expression ::= logic_or [ "=" expression ]`, with `=` right-associative and the lowest-precedence operator there is. That one change does more than let `x = y = 1` chain – it assigns `1` to `y`, then that same `1` to `x`, since an assignment now evaluates to the value it assigned, not to nothing. It also means the parser stops trying to decide, while parsing, whether a particular expression is an acceptable place to assign into: `1 = 2` and `f() = 5` both parse without complaint, the same rule applying uniformly to every expression. Evaluation is what actually decides – only

an identifier or an index/field chain not ending in a call can receive a value, so `f() = 5` still fails, just when it runs rather than when it is read. A chain ending in an index is still fine even when a call supplied the base – `f()[0] = 5` writes into the list `f()` just produced, since the outermost suffix there is the index, not the call.

Statements get the same kind of unification. Rather than special-casing assignment and a trailing call as the only two things an identifier-led line could be, any expression is now a legal statement on its own, its value computed and discarded: `1 + 1;` is legal and simply useless, the same as in most other languages. Vertex does not try to forbid statements that do nothing – a clean, predictable rule that happens to admit a few harmless lines nobody would write on purpose is worth more than a parser that has to guess which expressions are worth standing alone.

A statement beginning with `function` needs one extra token of lookahead to resolve, since the keyword alone is ambiguous: an identifier right after it means the named-definition sugar, statement-only syntax with no expression-level form; anything else means an ordinary anonymous function expression, used here as a statement. That covers an immediately invoked function, called the moment it is written:

```
function(a) { print(a * 2) }(5);
```

This prints `10`, with no parentheses needed around the function expression itself – it is already a factor, the same as any other, and the suffix chain's `(5)` is just a call applied to it.

Evaluation Order and Status

A call evaluates its callee, then each argument, left to right, checking status after each one before moving to the next – the same order every other multi-child node already uses. Only once every argument has a value does the call check arity: a status from an argument still escapes the call, even one that would have failed arity, the same way a status anywhere else is never masked by a later check.

Once arity passes, a new environment is built: `$PARENT` set to the environment active at the call site, one entry per parameter, bound to the matching argument. The function's body runs in that new environment, and its own resulting status decides what the call reports back:

Body's status	Call reports
`"return"`	The returned value, with no status: the call finished normally
`None` (fell off the end)	`null`, with no status
`"exit"`	Passed outward unchanged, the same as it passes outward through a loop
`"break"` or `"continue"`	A specific error: neither one has a loop left to belong to

That `"exit"` row is worth pausing on. `exit` itself still cannot sit inside a larger expression – that has not changed – but a call can, and a call whose body exits carries that status out through whatever expression the call sits inside, the same way any other status already does:

```
numbers = [10, 20, 30];
function stopAt(code) { exit(code); };
print(numbers[stopAt(5)]);
```

This exits with status `5`; the index is never used and the print never runs. An earlier chapter promised that function calls would eventually give a real way to exit from something nested, once a call existed to be nested in. This is that promise kept, and it needed no new code: the call node already checks its callee's and arguments' statuses the same way every other node does, and a function's body is evaluated by the same `evaluate()` recursion as anything else.

`return` is a status, propagated exactly the way `exit`, `break`, and `continue` already were: it travels up through blocks, `if`, and `while` until something is positioned to consume it. A call is the first structure built so far that consumes anything other than `break` and `continue`.

That last row is new. Earlier, a stray `break` or `continue` outside any loop simply propagated all the way to the program's own top level, where nothing was watching for it either. Now that a function call is also a boundary a `break` or `continue` can reach with nothing left to catch it, both boundaries – a call, and the top-level program itself – reject it explicitly:

```
function f() { break };
f();
```

This raises `'break' used outside a loop`, not a vague, unrecognized status. A `break` or `continue` used inside a loop that is itself inside a function body is unaffected: it is consumed by that loop exactly as before, long before it could ever reach the call boundary.

A bare `return` outside any function is rejected the same way, at the same program boundary:

```
return 5;
```

This raises `'return' used outside a function`, not the generic unhandled-status error every chapter's unrecognized statuses would otherwise hit. Two separate rules cover every case between them: a call consumes a `return` status silently, turning it into an ordinary value, the moment its body produces one; the program boundary has no call above it to do that consuming, so a bare `return` that reaches it is rejected outright, with its own check sitting right next to the one that already rejects a stray `break` or `continue` there.

Verification

`code/functions.v` is a runnable example covering definition, calling, local variables, recursion, dynamic scoping, and chained calls. Run it with `code/vertex code/functions.v` from this chapter's directory. `bash code/run-tests.sh` checks the interpreter against the inherited behavior and this chapter's new rules, including the specific error a stray `break` or `continue` now produces at a function or program boundary.

Considerations

1. `makeAdder(1)(2)` fails because the returned function cannot see `makeAdder`'s own local `a`. What would a function value need to retain, beyond its parameters and body, to make that example work?
2. Dynamic scoping resolves a free name by looking at the caller, not at wherever the function was written. What kind of bug would that make easy to introduce by accident, that lexical scoping would not?
3. This chapter lets a call's result be indexed or field-accessed (`f()[0]`, `f().name`) but never assigned to directly (`f() = 5`). Why does one make sense and the other does not?
4. A function call consumes `return` silently, turning it into an ordinary value, but the same call rejects `break` or `continue` outright instead of consuming those too. Why does it make sense to treat them differently?

Practice

1. Write a function `maxOf(a, b)` that returns whichever of its two arguments is larger.
2. Write a function `countDown(n)` that prints every whole number from `n` down to `1`, one per line, using a loop inside the function body, not recursion.
3. Write a recursive function `sumTo(n)` that returns the sum of every whole number from `1` to `n`. What does `sumTo(0)` return, and why?
4. Predict what `showGreeting()` prints on its own, called with no enclosing caller that ever assigned `greeting`, and explain why.
5. Write a function `isEmpty(items)` that returns whether a list has no elements, then use it as the condition of an `if` statement.

Answers to the Practice Questions

1. Something like:

```
function maxOf(a, b) {
    if (a > b) { return a };
    return b;
};
```

2. Something like:

```
function countDown(n) {
    i = n;
    while (i >= 1) {
        print(i);
        i = i - 1;
    };
};
```

3. Something like:

```
function sumTo(n) {
    if (n == 0) { return 0 };
    return n + sumTo(n - 1);
};
```

`sumTo(0)` returns `0`, the base case: it is the point where recursion stops calling itself and starts returning actual values back up the chain of waiting calls.

4. It raises `Unknown identifier: greeting`. Dynamic scoping looks for `greeting` by walking `$PARENT` links outward from wherever `showGreeting` was actually called. Called with nothing above it ever having assigned `greeting`, that walk reaches the top level and finds no binding at all.

5. Something like:

```
function isEmpty(items) { return length(items) == 0 };
if (isEmpty([])) { print("nothing here") };
```

References

1. Wikipedia contributors. “Scope (computer science).” Wikipedia. https://en.wikipedia.org/wiki/Scope_(computer_science)